

Fontes AI Admin Adjunta

Relatório técnico completo — estado atual do projeto

Campo	Valor
Data de geração	19/09/2026 07:36 UTC
Repositório	https://github.com/igor1fontes-Panda/ia-admin-adjunta.git
Branch / HEAD	main — b593e5f fix: exclude Freebuff isolate/ snapshot from vitest discovery
Objetivo do documento	Transferência completa de conhecimento: arquitetura, código, banco, auth, bots, CI/CD, testes e pendências — sem segredos, pronto para continuar o desenvolvimento em outro local

Este documento contém tudo o que está aplicado e ligado no projeto e como está ligado. Nenhuma credencial ou valor secreto é incluído — apenas nomes de variáveis e suas funções. Com este relatório + o repositório é possível reconstruir e evoluir o sistema em qualquer ambiente.

1. Visão geral

Produto: plataforma de administração com IA para PMEs em Angola/Portugal — captura e qualificação automática de leads, gestão de clientes e vendas, cobrança (AOA), agentes autônomos que aprendem com dados reais, academia de IA (AI Teacher que ensina os agentes), QA de entrega e pacotes de produto. Planes: Starter 1.250 Kz, Professional 2.916 Kz, Enterprise 8.333 Kz por mês.

Princípio central do produto: "REAL DATA ONLY" — nenhuma parte do sistema fabrica dados. Zeros são zeros reais; agentes sem dados dizem honestamente que não têm aprendido; evidências de produto trazem fonte, contagem e frescor.

Camada	Tecnologia	Estado
Frontend	React 19 + Vite 8 SPA, React Router 7, Tailwind 3, Framer Motion, Recharts, lucide-react, i18n próprio (PT/EN)	Produção pronta; build ok (2.4s)
API	Vercel Functions (api/*.ts) Node 22, TypeScript estrito, helper único de sessão (api/lib/http.ts)	Handlers reais verificados E2E (health 200, leads 201, guard 401)
Banco	Neon Postgres (PostgreSQL 18.6) + Drizzle ORM; 10 tabelas migradas via drizzle-kit push	MIGRADO E VERIFICADO no projeto Neon (10/10 tabelas)
Auth padrão	Better Auth auto-hospedado (sessão cookie HttpOnly, tabelas user/session/account/verification)	Implementado; verificação de e-mail exigida pelo modo gerenciado
Auth opcional	Clerk (aditivo) — @clerk/react + verificação JWT em todas as rotas; ativa somente com chaves presentes	Integrado e testado (76+ testes verdes); chaves dev accountless provisionadas
Bots/agentes	6 scripts Node (GitHub Actions cron) + ai_engine.py; logger JSON estruturado	Funcionais; 2 bugs de SQL corrigidos e testados
CI/CD	GitHub Actions (lint, typecheck, coverage gate 50%, testes Python, build, audits)	Ativo a cada push/PR
Hospedagem	Vercel (vercel.json: SPA + rewrites /api) — alternativa: Freebuff hosting	Deploy pendente de redeploy com DATABASE_URL (ver §12)

2. Arquitetura e como está ligado

Fluxo de dados em produção:

```
Navegador (SPA React em dist/) ■■ dados: fetch /api/leads /api/clients /api/orders /api/activity /api/agents /api/deliveries ■ → Vercel Function (api/*.ts) → Drizzle ORM → Neon Postgres (DATABASE_URL, pooled) ■■ auth: Better Auth – POST /api/auth/sign-in|sign-up → function api/auth/better-auth.ts ■ → cookie HttpOnly session_token → tabelas user/session no Neon ■ (alternativa com chaves: Clerk – Authorization: Bearer verificado em api/lib/http.ts) ■■ analytics: @vercel/analytics + speed-insights GitHub Actions (cron) ■■ 06:00 UTC diário: lead-hunter (qualificador), growth-marketing, teacher-agent, ops-manager ■■ hourly: error-handler; skills-scout; insight (python3 ai_engine.py) ■■ cada bot → scripts/bot-lib.mjs (shim supabase-js → Neon HTTP driver) → agent_memory / activity_log AI providers dos bots: Gemini (cadeia de fallback de modelos) → Blackbox (OpenAI-compatible) → null. Sem chave, o bot roda em modo determinístico sobre dados reais – nunca simula.
```

Fonte única de verdade do schema: db/schema.ts (Drizzle). O catálogo espelha as migrações Supabase 0001..0005 uma-a-uma, mais as 4 tabelas do Better Auth. **Aplicação no Neon:** já executada (drizzle-kit push), confirmada por information_schema e por smoke (10 tabelas presentes).

3. Estrutura de código completa

Total: **82 arquivos-fonte**, ~10,140 linhas (TS/TSX/MJS/PY, sem node_modules). Diretórios por volume:

Diretório	Arquivos	Linhas
.agents/skills/supabase-postgres-best-practices/	2	137
.agents/skills/supabase-postgres-best-practices/references/	34	1857

Diretório	Arquivos	Linhas
.claude/skills/neon/	1	474
.claude/skills/neon/references/	6	282
.claude/skills/neon-postgres/	1	279
.claude/skills/neon-postgres/references/	3	326
.github/	1	12
.github/workflows/	7	571
.tools/	2	510116
agent/skills/supabase-postgres-best-practices/	2	153
agent/skills/supabase-postgres-best-practices/references/	34	1857
api/	7	397
api/auth/	1	32
api/lib/	1	73
api/stripe/	2	173
db/	2	211
docs/	1	136
lib/	1	29
lib/db/	2	62
lib/db/migrations/	1	45
public/	1	5
scripts/	19	3091
server/	2	112
src/	12	1317
src/app/	1	14
src/app/api/auth/[...all]/	1	4
src/components/	7	480
src/lib/	10	1009
src/lib/i18n/	3	1124
src/views/	3	796
src/views/dashboard/	3	57
src/views/dashboard/tabs/	9	1488
tests/	1	185

Maiores arquivos (todos < 522 LOC após os refactorings):

Arquivo	LOC
src/lib/engine.ts	522
src/lib/i18n/pt.ts	501
src/lib/i18n/en.ts	485
scripts/generate-project-report.py	438
scripts/bot-lib.mjs	414
src/engine.test.ts	380
src/views/dashboard/tabs/AgentsTab.tsx	296

Arquivo	LOC
src/views/Dashboard.tsx	291
src/views/Landing.tsx	283
src/lib/data.ts	252
scripts/ops-manager.mjs	252
src/engine.pack.test.ts	236
src/views/dashboard/tabs/OverviewTab.tsx	234
scripts/growth-marketing.mjs	233

src/views/Dashboard.tsx é um shell de 291 LOC (estado + fetch + troca de abas); cada aba vive em src/views/dashboard/tabs/*.tsx (<300 LOC cada). O dicionário i18n vive em src/lib/i18n/pt.ts e en.ts (pt é fonte da verdade; teste de paridade impede divergência).

4. Banco de dados — 10 tabelas (Neon, migrado)

Negócio (espelham migrações Supabase; PK uuid, created_at timestampz):

Tabela	Colunas principais	Índices/FKs
leads	company, contact_name, email, niche(SaaS), channel(website), score int=50, status(new), ai_action	idx created_at
clients	name, email, plan(starter), mrr numeric(12,2), status(active)	idx created_at
orders	client_id→clients(set null), client_name, amount numeric, currency(AOA), method(multicaixa), status(pending), reference	idx created_at + UNIQUE reference
activity_log	kind(system/bot), message	idx created_at
agent_memory	agent, key, value jsonb, updated_at	UNIQUE (agent,key) — upsert de aprendizado
delivery_status	client_id→clients, order_id→orders(cascade), pack, method, amount, qa_status(pending/passed/failed), checks jsonb, notes, verified_at	idx order_id, qa_status

Better Auth (geradas pelo schema do próprio Better Auth):

Tabela	Colunas principais
user	id text, name, email (UNIQUE), email_verified bool, image, created/updated_at
session	id, user_id→user(cascade), token (UNIQUE), expires_at, ip_address, user_agent
account	id, user_id→user(cascade), account_id, provider_id, access/refresh/id tokens, password (hash), scope
verification	id, identifier, value, expires_at (verificação de e-mail / reset)

Ferramentas: drizzle.config.ts (dialect postgresql, DATABASE_URL|NEON_DATABASE_URL) • npm run db:push (drizzle-kit push) • npm run db:smoke (scripts/db-smoke.mjs — mapa de linhas por tabela + versão do Postgres) • npm run db:neon:health. Migrações SQL legadas em lib/db/migrations/0001_better_auth.sql e supabase/migrations/ (referência histórica — Neon é o alvo atual).

5. API (Vercel Functions)

Rota	Métodos	Auth	Função
/api/health	GET	pública	ok + database.configured/connected + auth.configured (usada pelo health check)
/api/leads	GET/POST	GET requer sessão; POST público (formulário da landing)	listar / capturar lead (defaults forçados: status=new, score=50)
/api/clients	GET/POST	requer sessão	CRUD de clientes por plano
/api/orders	GET/POST/PATCH	requer sessão	vendas; marcar pago cria delivery_status
/api/activity	GET	requer sessão	feed de atividade (bots + sistema)
/api/agents	GET	requer sessão	agent_memory consolidada
/api/deliveries	GET/PATCH	requer sessão	QA de entrega (checks, aprovação)
/api/auth/better-auth	*	pública	Better Auth montado (sign-in/up/out, get-session, send-verification-email)
/api/stripe/create-checkout-session	POST	sessão	checkout (inerte sem STRIPE_SECRET_KEY)
/api/stripe/webhook	POST	assinatura	webhook Stripe (verificação de assinatura)

Toda rota autenticada passa por getSessionUser (api/lib/http.ts): 1) cookie Better Auth → 2) fallback Bearer JWT do Clerk (server/clerk.ts, verifyToken standalone) → 401 se nenhuma. Respostas de erro padronizadas por errorMessage().

6. Frontend

Rota	View	Notas
/	Landing	hero, packs/planos, formulário público de lead (POST /api/leads), prova social
/auth	Auth	Better Auth (default) OU componentes SignIn/SignUp do Clerk quando chaves presentes; painel de verificação de e-mail com reenvio (PT/EN)
/dashboard	Dashboard (protegido — RequireAuth)	shell + 9 abas: Overview, Product packs, Leads, Analytics, Clients, Orders, AI Agents, Ecosystem, Ops
*	redirect → /	—

Providers em cascata (App.tsx): PreferencesProvider (tema claro/escuro + locale persistidos) → SessionProvider (clerk-bridge: Clerk se configurado, senão Better Auth — mesma forma de sessão) → BrowserRouter. RequireAuth redireciona para /auth preservando o destino. Realtime via polling/subscriptions em src/lib/data.ts; mock mode (VITE MOCK_DATA=true) fornece fixtures determinísticas para dev.

7. Autenticação — 3 camadas

Camada	Estado	Como está ligada
Better Auth (self-hosted) — PADRÃO	ativo	server/auth.ts → Drizzle adapter → tabelas user/session/account/verification no Neon; cookie HttpOnly; segredo BETTER_AUTH_SECRET; e-mail+senha (min 8); verificação exigida quando provedor de e-mail configurado
Clerk — OPCIONAL/ADITIVO	integrado, aguardando claim da app	src/lib/clerk-bridge.tsx comuta o provider; Vite expõe NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY (pública) além de VITE_*; server/clerk.ts valida Bearer JWT em todas as rotas via CLERK_SECRET_KEY; chaves dev accountless em .env.local (geradas por npx clerk init + env pull)
Neon Auth (Better Auth gerenciado) — DO PROJETO NEON	ativo no Neon, NÃO consumido pelo app	host ep-broad-salad-zanhmhy8...neonauth.../neondb/auth responde /ok 200 e JWKS válido; CORS aberto com credenciais; origins confiáveis: localhost:5173 e :3000. O app não depende dele — manter como opção futura

8. Bots / agentes autônomos

Bot (script)	Cadência	O que faz (dados reais apenas)
lead-hunter.mjs — Lead Qualifier	diária 06:00 UTC	lê leads novos → pontua (Gemini/Blackbox ou regra determinística) → grava score/ai_action; aprende viés por canal em agent_memory (channel_bias)
error-handler.mjs	hourly	triage de incidentes; acumula known_fixes em agent_memory
growth-marketing.mjs	diária	diagnóstico do funil (bottleneck) → UMA campanha (audiência, canal, métrica, kill criteria); nunca envia nada
teacher-agent.mjs — AI Teacher	diária	estuda o mercado real e escreve market_brief na memória de cada aluno (academia: enrolled→in_training→trained)
ops-manager.mjs — AI Manager	diária	missões por agente (agent_memory key=mission) + QA de deliveries
skills-scout.mjs	diária	mapeia skill_entry (ecossistema skills.sh)
ai_engine.py — Insight Engine	diária (bot:insight)	lê métricas reais → insight (Gemini→Blackbox→regra) → activity_log; briefing de voz opcional via Hume (scripts/hume_voice.py)

Infra compartilhada (scripts/bot-lib.mjs): shim supabase-js sobre o driver HTTP do Neon (primary) com fallback legado Supabase; askAI com cadeia de modelos; parseJSONArray/Object; diagnoseSupabaseError (senhas/relação ausente/rede → conselho acionável); dbRemember/dbRecall (agent_memory). **Logging:** JSON por linha {level, ts, bot, msg} via createLogger — todos os 6 bots já carimbam o próprio nome. **Testes:** scripts/bot-lib.test.mjs (13 testes, driver mockado, zero rede) provam o SQL gerado; 2 bugs reais corrigidos (update com AND, maybeSingle com []).

9. Variáveis de ambiente (nomes — nenhum valor aqui)

Referência viva: README.env-vars.md. Onde obter cada valor está no README.

Grupo	Variáveis	Onde vive
Públicas (SPA)	VITE_SUPABASE_URL, VITE_SUPABASE_PUBLISHABLE_KEY (ou VITE_SUPABASE_ANON_KEY), VITE MOCK_DATA, VITE_CLERK_PUBLISHABLE_KEY / NEXT_PUBLIC_CLERK_PUBLISHABLE_KEY	build do Vite
Neon (primário)	DATABASE_URL (pooled), NEON_DATABASE_URL	server: .env.local, deploy env, GitHub secrets
Better Auth	BETTER_AUTH_SECRET (ou AUTH_SECRET), SITE_URL	idem
Clerk backend	CLERK_SECRET_KEY, CLERK_ISSUER	idem (opcional)
Supabase legado (bots fallback)	SUPABASE_URL_2/SUPABASE_URL, SUPABASE_SECRET_KEY/SUPABASE_SERVICE_ROLE_KEY, SUPABASE_PUBLISHABLE_KEY_2, SUPABASE_JWKS_URL	GitHub secrets
AI	GEMINI_API_KEY, GEMINI_MODEL, BLACKBOX_API_KEY, BLACKBOX_BASE_URL, BLACKBOX_MODEL	GitHub secrets
Voz	HUME_API_KEY	GitHub secrets
Stripe	STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET	deploy env
Scripts	AUTH_SMOKE_PASSWORD, AUTH_SMOKE_URL	.env.local (opcional)

10. CI/CD (.github/workflows)

Workflow	Gatilho	Passos-chave
ci.yml	push main / PR / manual	npm ci → py_compile bots → unittest Python (tests/) → pip-audit → npm audit(high) → lint → typecheck → coverage (gate 50% lines) → build → grep de segredos no bundle → resumo
ai-bots.yml	cron 06:00 UTC + hourly	executa os 6 bots + insight com secrets reais

Workflow	Gatilho	Passos-chave
deploy.yml	push main	deploy Vercel (vercel.json: build npm run build → dist/, rewrites /api e SPA)
supabase-migrations.yml	merge que mude supabase/migrations/	supabase db push (histórico; Neon é o alvo atual)
deploy-status.yml / preview-environment.yml / stale.yml	eventos	saúde do deploy, preview, limpeza de issues
dependabot.yml	semanal	atualizações npm + pip

11. Testes e qualidade (estado verificado)

Suíte	Arquivos	Testes	Notas
Vitest (jsdom)	11	69/69 ✓	engine (38) · bot-lib shim (13, zero rede) · auth-flow (4) · i18n paridade+prefs (5) · data/db-null-safety (7) · dashboard-nav (1) · errors (1)
Python unittest	1	8/8 ✓ (0,007s)	ai_engine: compute_stats, recall_learning, REST supabase, fallback Gemini — requests fake, sem credenciais
Coverage gate	—	71.9% lines · 60.4% branches	thresholds 50/35/50/40 ENFORÇADOS no CI (exit ≠ 0 abaixo)
Lint/Typecheck	—	0 erros	eslint src · tsc -b --noEmit (strict)
Build	—	✓ 2.4s	vite build → dist/ (~474 KB gzip 146 KB)
E2E local	—	gauntlet 5/5 ✓	scripts/e2e-gauntlet.mjs + dev-server.mjs: SPA + handlers reais + Neon (health 200, lead 201 persistido, guard 401)

Nota: o snapshot de build da plataforma (isolate/) é espelhado e foi excluído do discovery do Vitest (vitest.config.ts exclude) para não duplicar a suíte.

12. Estado atual e pendências (o que falta ligar)

Item	Estado	Ação pendente
Neon — schema	■ 10/10 tabelas migradas (drizzle-kit push, PostgreSQL 18.6)	nada
Neon — segredo no sandbox	■ em .env.local (injetado nos shells/preview)	nada
Vercel — produção	■ último deploy falhou (DEPLOYMENT_NOT_FOUND); DATABASE_URL ausente no ambiente de deploy	adicionar DATABASE_URL (pooled) nas env vars do deploy + redeploy; após isso /api/health responde 200 e o dashboard entra em Live
Neon Auth — trusted origins	localhost ok; domínio Vercel não	adicionar o domínio de produção em Neon Console → Auth → Trusted Origins (se/quando o login usar o Neon Auth)
Clerk — claim da app accountless	chaves dev funcionam (mock/dev)	npx clerk auth login → npx clerk deploy; depois trocar chaves dev por produção e CLERK_SECRET_KEY no deploy
Stripe	rotas existem, inerte sem chave	definir STRIPE_SECRET_KEY/WEBHOOK quando for cobrar
Legados	next.config.ts, src/app/, supabase/, Dockerfile, docker-compose	limpeza opcional — não afetam o build Vite/CI
E-mail de verificação	Neon Auth envia (status true) no modo gerenciado	Better Auth self-hosted precisa de provider (Resend etc.) se a verificação for exigida sem Neon Auth

13. Git — últimos 14 commits (main)

```
b593e5f fix: exclude Freebuff isolate/ snapshot from vitest discovery
567aef0 docs: real contributing guide with AI-agent authorship convention
ab8d7fd ci: enforce coverage thresholds and run python bot logic tests
ade1107 test: ai_engine.py logic via fake requests, no live account
1430b63 test: zero-network bot-lib shim coverage; fix two real SQL bugs
8789247 feat: structured JSON logging for bots with per-script bot field
7c05140 chore: env var from auth smoke, complete env var docs, untrack coverage
c27bfc4 refactor: split Dashboard into focused tab modules under 300 LOC
a3d5e5a refactor: split i18n into dictionary modules with key-parity tests
18f556c test: pin engine product-pack, prompt-plan and scoring boundaries
3973453 feat: additive Clerk auth (bridge provider + backend token verification)
2d74335 feat: local full-stack preview server + e2e gauntlet
2832cb2 feat: managed Neon Auth sign-in flow with email verification
24226b7 Merge pull request #61 from igor1fontes-Panda/v0/fix-vercel-ci-errors
```

Convenção (CONTRIBUTING.md): 1 feature/fix por commit + teste pareado; autoria de agente registrada no trailer (Co-Authored-By) e revisão humana obrigatória antes de merge em main.

14. Como continuar o desenvolvimento em outro local

Passo a passo mínimo (qualquer máquina com Node 22+ e Python 3):

```
git clone https://github.com/igor1fontes-Panda/ia-admin-adjunta.git cd ia-admin-adjunta npm ci cp
.env.example .env.local # preencher no mínimo: # DATABASE_URL = connection string POOLED do Neon #
BETTER_AUTH_SECRET (openssl rand -base64 32) # SITE_URL=http://localhost:5173 npm run db:push #
aplica o schema db/schema.ts no Neon npm run dev # SPA :5173 # API real local (handlers Vercel +
SPA em uma porta): npm run preview:fullstack # Prova E2E (health/leads/guard/session): npm run e2e
# Smoke do banco (linhas por tabela): npm run db:smoke # Deploy: vercel.json já pronto (build →
dist/, rewrites /api/*). # Setar DATABASE_URL, BETTER_AUTH_SECRET, SITE_URL (+ opcionais) no painel
do host. # Alternativa Vercel: qualquer host estático + as functions de api/ adaptadas, # ou
Freebuff hosting (build = npm run build, saída dist/).
```

O que já está provado funcionando de ponta a ponta: formulário público → /api/leads → Drizzle → Neon → linha persistida → dashboard; login Better Auth com verificação de e-mail; 6 bots com logger JSON e memória de aprendizado; CI completo com gate de cobertura. O único elo pendente de produção é o redeploy com DATABASE_URL

(§12) — código, schema, testes e CI estão verdes e prontos.